

SILENTSENSE

An IoT-Based Inactivity and Emergency Panic Monitoring System for Elderly and Child Care

A B.Tech Project Report Submitted in Partial Fulfillment of the
Requirements for the Degree of

Bachelor of Technology in Computer Engineering

Submitted By:

Ammara Fairoz Sirkhot

Team Name: AFS_Synaptic_Edge

Department of Computer Science Engineering

Academic Session: 2025–2026

Acknowledgements

I would like to express my sincere gratitude and deep appreciation to our project supervisor and the **Department of Computer Engineering** for providing the opportunity, guidance, and resources required to carry out this project.

I am thankful to my team **AFS_Synaptic_Edge** for their dedication, technical insights, and collaborative effort throughout the development of the **SilentSense** system. Lastly, I extend my heartfelt thanks to my family and peers for their continuous motivation and support during the realization of this work.

Ammara Fairoz Sirkhot

(Team: AFS_Synaptic_Edge)

Department of Computer Engineering

Abstract

In modern households, ensuring the continuous safety of vulnerable individuals—such as elderly persons living alone or unattended young children—presents a critical challenge. Sudden falls, medical emergencies, or prolonged physical distress often lead to periods of prolonged immobility where the individual is unable to seek help.

This report presents **SilentSense**, a hybrid IoT-embedded monitoring framework designed to continuously monitor room activity and provide instant emergency response. The system integrates an Arduino micro-controller running an IPC (Inter-Process Communication) Bridge layer with a Linux-based Python runtime environment. Using a Passive Infrared (PIR) sensor and a Panic Push Button, the hardware monitors human movement and physical distress signals.

When movement stops for an extended duration (30 seconds in prototype configuration) or when the manual Panic Button is pressed, the system immediately triggers a localized audiovisual signal (LED/Buzzer) and dispatches real-time structured alerts to caregivers via the Telegram Bot API. SilentSense provides a low-cost, non-intrusive, privacy-preserving, and scalable solution for smart home safety monitoring.

Contents

Acknowledgements	1
Abstract	2
1 Introduction	7
1.1 Overview & Motivation	7
1.2 Problem Statement	7
1.3 Project Objectives	8
1.4 Scope of the Report	8
2 Literature Survey	9
2.1 Background Review	9
2.2 Comparative Analysis	9
3 Requirement Analysis	11
3.1 Problem Definition	11
3.2 System Modules and Functionalities	11
4 System Design	13
4.1 System Architecture	13
4.2 Control Flow Diagrams	13
4.3 UML Diagrams	13
4.3.1 Use Case Diagram	13
4.3.2 Sequence Diagram	15
5 Software & Hardware Requirements	16
5.1 Hardware Specifications	16
5.2 Software Specifications	16
6 Coding & Implementation	18
6.1 Embedded C++ Firmware (Arduino Bridge Layer)	18
6.2 Python Application Layer (python/main.py)	20

6.3	Functionality and Parameter Matrix	23
7	Conclusion & Future Scope	24
7.1	Conclusion	24
7.2	Future Scope	24
	Bibliography	25

List of Figures

4.1	SilentSense High-Level System Architecture Diagram	13
4.2	System Control Flow Diagram (CFD)	14
4.3	UML Use Case Diagram for SilentSense	14
4.4	UML Sequence Diagram for Alarm & Notification Flow	15

List of Tables

2.1	Comparative Study of Existing Fall/Inactivity Monitoring Systems . . .	10
5.1	Hardware Component Requirements	16
5.2	Software Tools and Environment Specifications	17
6.1	Embedded C++ Method Specifications	23
6.2	Python Application Module Functions	23

Chapter 1

Introduction

1.1 Overview & Motivation

With an increasing aging population and a growing number of nuclear families where both parents work, ensuring the safety of elderly individuals and children at home has become a major concern. Incidents such as sudden fainting, strokes, slips and falls, or child accidents often leave the victim immobilized and unable to reach a smartphone or phone landline to make a call.

Traditional home security systems focus primarily on intrusion detection rather than internal health and safety monitoring. Camera-based indoor monitoring systems present serious privacy concerns and high power consumption. **SilentSense** addresses this gap by creating an autonomous, non-intrusive ambient sensor framework that tracks physical presence via infrared light variation and offers an instantaneous one-touch manual panic trigger.

1.2 Problem Statement

Existing health emergency systems often require individuals to wear smartbands or pendants continuously, which elderly individuals often forget to charge or wear. Furthermore, video monitoring inside private rooms causes discomfort and privacy violations. There is a need for a non-wearable, privacy-friendly, low-power, and affordable system capable of:

1. Detecting prolonged physical inactivity in living spaces.
2. Providing a manual emergency override (Panic Button).
3. Activating immediate local alarms while notifying caregivers remotely via instantaneous push messaging (Telegram).

1.3 Project Objectives

- Design and deploy an embedded system using an **Arduino Bridge** model combining real-time GPIO performance with Python networking.
- Interface a Passive Infrared (PIR) motion sensor to detect ambient movement and identify prolonged stillness.
- Implement a dedicated hardware Panic Button with internal pull-up logic for instant emergency signaling.
- Integrate an automated Telegram Bot API alerting mechanism over Wi-Fi for remote notification delivery.
- Ensure low latency, high reliability, and privacy compliance without using visual cameras.

1.4 Scope of the Report

This report covers the design, hardware setup, software integration, UML modeling, code implementation, and functional evaluation of the **SilentSense** monitoring framework developed by team **AFS_Synaptic_Edge**.

Chapter 2

Literature Survey

2.1 Background Review

Inactivity and emergency monitoring systems have evolved from simple medical alert buttons to complex smart home networks.

1. **Wearable Accelerometer-based Systems:** Devices attached to wrists or belts monitor acceleration vectors to detect falls. While effective for fall detection, they suffer from high false-positive rates due to sudden movements and rely on user compliance in wearing the device daily.
2. **Vision-based Camera Monitoring:** Deep learning models process live video streams to identify abnormal human postures. Despite high accuracy, these systems are expensive, require significant computing power, and violate user privacy in spaces like bedrooms or bathrooms.
3. **Ambient PIR & Contact Sensors:** PIR sensors detect human motion by measuring changes in infrared radiation levels emitted by surrounding objects. Combining PIR sensors with edge microcontrollers offers a balanced approach: zero invasion of privacy, extremely low power draw, and easy installation.

2.2 Comparative Analysis

Table 2.1 summarizes the comparative performance of existing monitoring frameworks against the proposed SilentSense architecture.

Table 2.1: Comparative Study of Existing Fall/Inactivity Monitoring Systems

Parameter / Feature		Vision-Based Systems		Wearable Pen-dants		SilentSense (Proposed)
Privacy Preservation		Low (Video not recorded)	Feed	High		High (No imagery)
User Maintenance		Low		High (Daily charging)		Zero (Stationary)
Manual Trigger	Panic	No		Yes (Button on device)		Yes (Dedicated Switch)
Deployment Cost		High		Moderate		Low / Affordable
Network Mechanism		Local / Cloud	Server /	Cellular / tooth	Blue-	IPC Bridge + Telegram

Chapter 3

Requirement Analysis

3.1 Problem Definition

The system must monitor a room occupied by a vulnerable person (elderly/child). The dual failure conditions to detect are:

- **Active Distress:** The user experiences sudden pain or distress and manually hits the Panic Switch.
- **Passive Distress / Immobility:** The room transitions from active movement to absolute stillness for longer than a predefined threshold (e.g., 30 seconds during testing/demonstration phase), indicating potential unconsciousness or incapacity.

Upon encountering either condition, the system must trigger local visual/acoustic indicators and publish an emergency log message to a Telegram chat accessible by caregivers.

3.2 System Modules and Functionalities

The project is structured into four functional modules:

1. **Sensor Hardware & Controller Module (C++):** Configures digital input/output pins for PIR sensor (D7), Panic Button (D2), and LED/Buzzer (D8). Manages debouncing and non-blocking timing using `millis()`.
2. **Bridge Communication Interface:** Uses `Arduino_RouterBridge.h` to bind C++ state provider functions (`get_motion`, `get_panic`) to an Inter-Process Communication (IPC) bus.
3. **Python Application Controller (`main.py`):** Periodically queries the Bridge layer at 200 ms intervals, tracks state shifts, and applies alert throttling (e.g., 10-second cooldown).

4. **Telegram Alerting Gateway:** Formats event data into Markdown-formatted alerts and dispatches payloads asynchronously over HTTPS to the Telegram Bot API Endpoint.

Chapter 4

System Design

4.1 System Architecture

Figure 4.1 illustrates the high-level architecture of the SilentSense system.

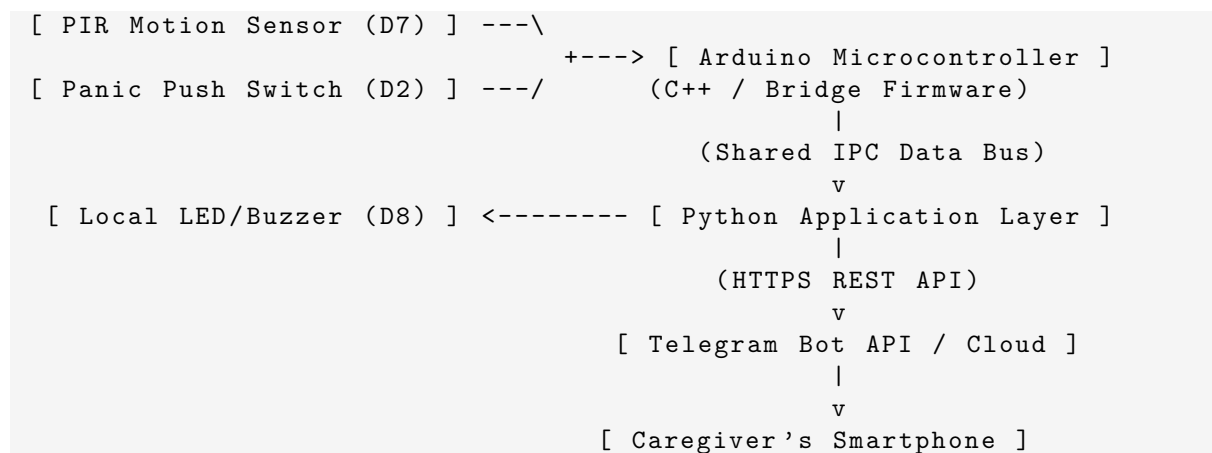


Figure 4.1: SilentSense High-Level System Architecture Diagram

4.2 Control Flow Diagrams

Figure 4.2 shows the complete decision logic and operational control flow of the monitoring system.

4.3 UML Diagrams

4.3.1 Use Case Diagram

Figure 4.3 details the primary operational use cases and external actor interactions.

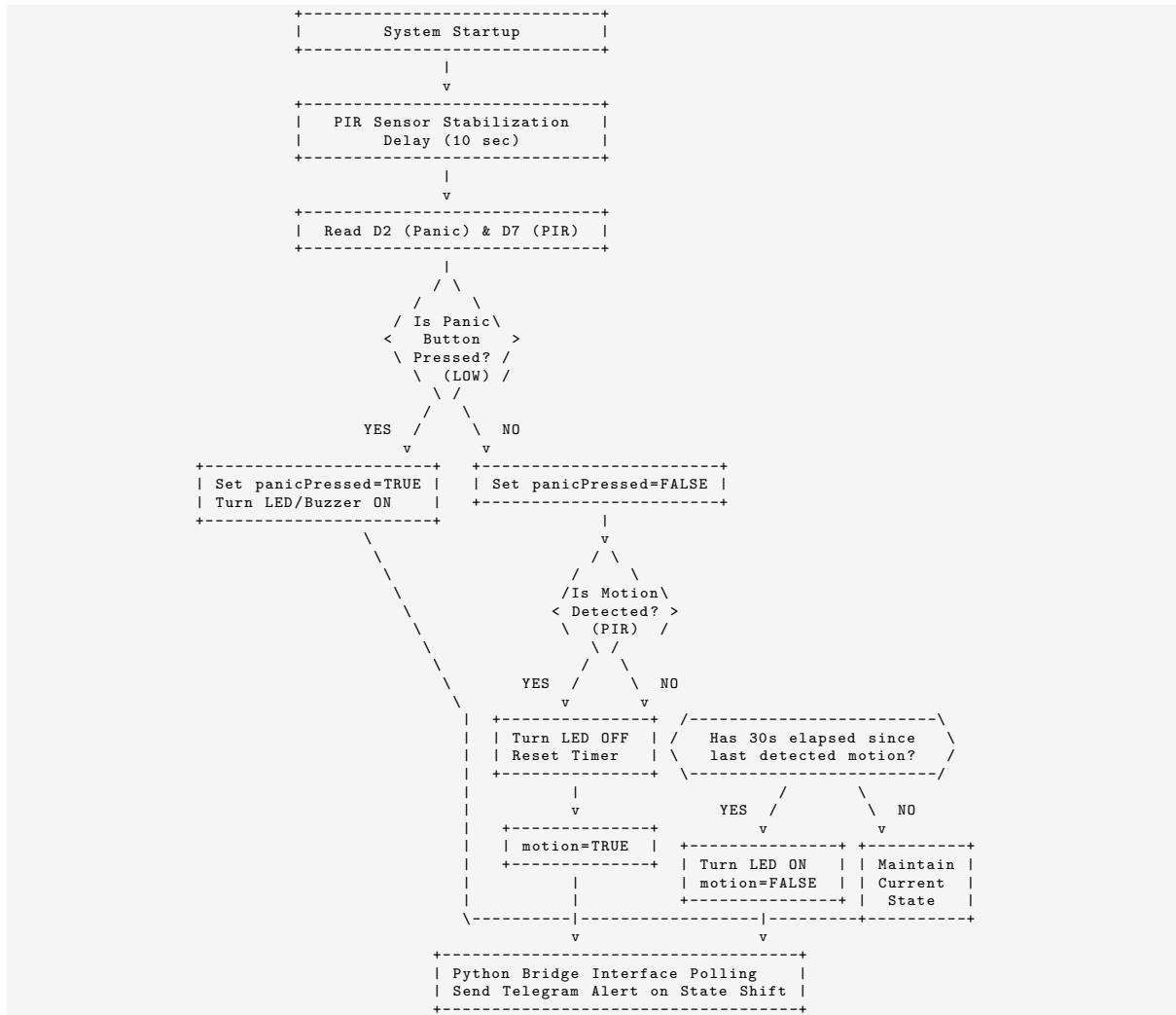


Figure 4.2: System Control Flow Diagram (CFD)

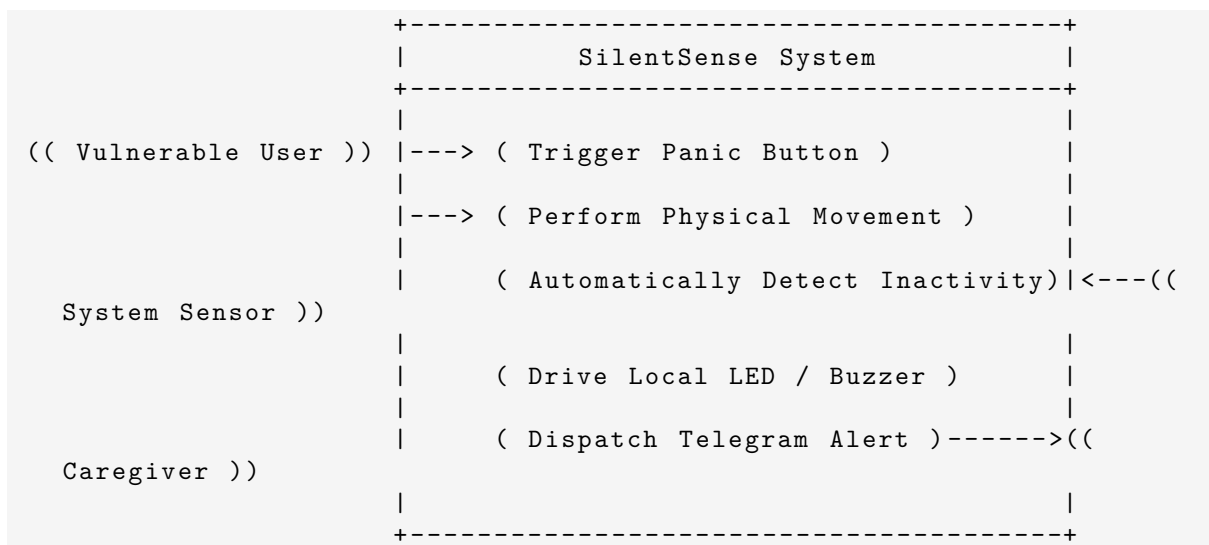


Figure 4.3: UML Use Case Diagram for SilentSense

4.3.2 Sequence Diagram

Figure 4.4 illustrates the message sequence flow during emergency button activation.

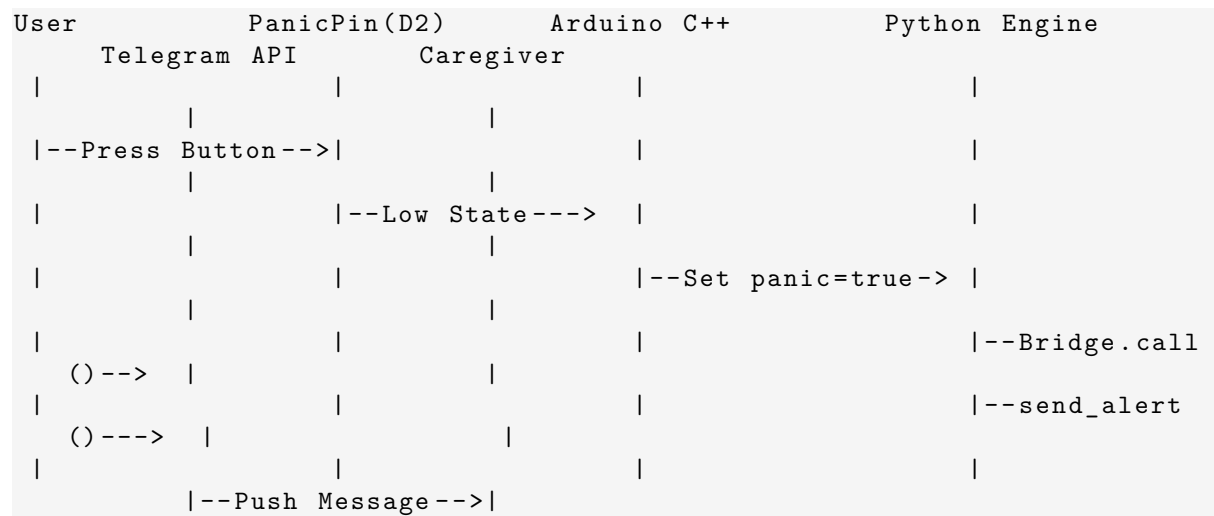


Figure 4.4: UML Sequence Diagram for Alarm & Notification Flow

Chapter 5

Software & Hardware Requirements

5.1 Hardware Specifications

The hardware bill of materials and component specifications are outlined in Table 5.1.

Table 5.1: Hardware Component Requirements

Component	Specification Model	/	Pin / Interface	Functional Purpose
Microcontroller	Arduino Router Bridge Board		Onboard Bus / GPIO	Core Processing & IPC
PIR Sensor	HC-SR501 Infrared Sensor		Signal → D7, VCC → 5V	Movement Detection
Panic Switch	Tactile Momentary Switch		Pin D2 → GND	Manual Panic Override
Alarm Unit	LED Light / Piezo Buzzer		Positive → D8, GND	Local Visual/Auditory Alert
Interconnects	Jumper Wires / Breadboard		Standard Dupont	Hardware Interconnects

5.2 Software Specifications

The complete software toolchain and environment details are listed in Table 5.2.

Table 5.2: Software Tools and Environment Specifications

Software Layer	Environment / Tool	Version / Library	Description
Embedded Firmware	C / C++	Arduino IDE v2.x	Hardware Control Firmware
Bridge IPC Library	Arduino_RouterBridge	Native Router SDK	C++ to Python IPC
Application Runtime	Python Runtime Environment	Python 3.9+	High-level Logic Engine
Networking Library	Python requests	HTTP/1.1 Requests	Telegram API Comm.
Messaging Platform	Telegram Messenger	Bot API	Cloud Alert Delivery

Chapter 6

Coding & Implementation

6.1 Embedded C++ Firmware (Arduino Bridge Layer)

The embedded C++ firmware handles physical hardware interfacing, pin interrupts, and IPC state exposures.

```
1 #include "Arduino_RouterBridge.h"
2
3 // Hardware Pin Assignments
4 int pirPin = 7;           // PIR Sensor OUT connected to Digital Pin
                             D7
5 int ledPin = 8;           // LED / Buzzer module connected to
                             Digital Pin D8
6 int panicPin = 2;         // Panic Push Button connected to Pin D2 (
                             Active LOW)
7
8 // Time tracking and State Flag Variables
9 unsigned long lastMotionTime = 0;      // Holds timestamp of last
                             motion event
10 const unsigned long delayTime = 30000; // 30 seconds threshold
                             for inactivity
11 bool motionDetected = false;           // Tracks present motion
                             status
12 bool panicPressed = false;             // Tracks panic button
                             activation state
13
14 /**
15  * @brief Exposes motion status to Python container
16  * @return int 1 if motion detected, 0 if inactive
17  */
18 int get_motion() {
```

```

19     return motionDetected ? 1 : 0;
20 }
21
22 /**
23  * @brief Exposes panic button status to Python container
24  * @return int 1 if panic button pressed, 0 otherwise
25  */
26 int get_panic() {
27     return panicPressed ? 1 : 0;
28 }
29
30 void setup() {
31     // Configure Hardware Pin Modes
32     pinMode(pirPin, INPUT);
33     pinMode(panicPin, INPUT_PULLUP); // Active low with internal
34                                     // pullup resistor
35     pinMode(ledPin, OUTPUT);
36     digitalWrite(ledPin, HIGH);      // Default state: LED ON (
37                                     // Indicates initial system boot / idle)
38
39     // Establish Bridge IPC communication with Python container
40     Bridge.begin();
41     Bridge.provide_safe("get_motion", get_motion);
42     Bridge.provide_safe("get_panic", get_panic);
43
44     // Wait 10 seconds to stabilize PIR sensor pyroelectric
45     // elements
46     delay(10000);
47 }
48
49 void loop() {
50     // 1. Evaluate Panic Button State
51     if (digitalRead(panicPin) == LOW) { // Button shorted to GND
52         panicPressed = true;
53         digitalWrite(ledPin, HIGH);      // Instantly activate
54                                         // visual alarm/buzzer
55     } else {
56         panicPressed = false;
57     }
58
59     // 2. Evaluate Motion Sensor Logic

```

```

56  int pirVal = digitalRead(pirPin);
57
58  if (pirVal == HIGH) {                                // Motion detected
59      digitalWrite(ledPin, LOW);                        // Deactivate indicator
        light during motion
60      lastMotionTime = millis();                        // Reset inactivity timer
61      motionDetected = true;
62  }
63  else {                                                // Motion absent
64      if (motionDetected && (millis() - lastMotionTime >= delayTime
        )) {
65          digitalWrite(ledPin, HIGH);                  // Activate alarm after 30s
        inactivity
66          motionDetected = false;
67      }
68  }
69
70  delay(100);                                          // Loop polling delay
71 }

```

Listing 6.1: Arduino C++ Firmware Implementation

6.2 Python Application Layer (python/main.py)

The Python application queries the bridge and transmits notification requests to the Telegram Bot API endpoint.

```

1  import time
2  import os
3  import requests
4  from arduino.app_utils import Bridge
5
6  # --- TELEGRAM BOT CREDENTIALS ---
7  TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "
        YOUR_BOT_TOKEN_HERE")
8  TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "
        YOUR_CHAT_ID_HERE")
9
10 # Global state trackers
11 last_motion_state = None
12 last_panic_state = False
13 last_alert_time = 0

```

```

14
15 def send_telegram_alert(message_text):
16     url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/
sendMessage"
17     payload = {
18         "chat_id": TELEGRAM_CHAT_ID,
19         "text": message_text,
20         "parse_mode": "Markdown",
21     }
22
23     try:
24         response = requests.post(url, json=payload, timeout=5)
25         if response.status_code == 200:
26             print("Telegram alert sent successfully!", flush=True
27         )
28         else:
29             print(f"Telegram alert failed: {response.text}",
30             flush=True)
31     except Exception as e:
32         print(f"Telegram error: {e}", flush=True)
33
34 def main():
35     global last_motion_state, last_panic_state, last_alert_time
36
37     print("=====", flush=
38     True)
39     print("SILENTSENSE SYSTEM STARTED", flush=True)
40     print("Monitoring PIR & Panic Button...", flush=True)
41     print("=====", flush=
42     True)
43
44     while True:
45         try:
46             # Query status variables from C++ Bridge
47             motion_detected = bool(Bridge.call("get_motion"))
48             panic_pressed = bool(Bridge.call("get_panic"))
49
50             # --- 1. PANIC BUTTON EVENT TRIGGER ---
51             if panic_pressed and not last_panic_state:
52                 now_str = time.strftime('%H:%M:%S')
53                 panic_message = (

```

```

50         f"*EMERGENCY PANIC ALERT!*\n\n"
51         f"*Time:* {now_str}\n"
52         f"*Status:* Panic Button Pressed!\n"
53         f"*Action:* Light / Buzzer Turned ON!"
54     )
55     print("PANIC BUTTON PRESSED!", flush=True)
56     send_telegram_alert(panic_message)
57     last_panic_state = True
58 elif not panic_pressed:
59     last_panic_state = False
60
61     # --- 2. PIR INACTIVITY SENSOR TRIGGER ---
62     if last_motion_state is None:
63         last_motion_state = motion_detected
64
65     if motion_detected != last_motion_state:
66         if motion_detected:
67             print("Motion Detected! LED turned OFF.",
flush=True)
68         else:
69             print("No motion for 30s! LED turned ON.",
flush=True)
70             current_time = time.time()
71
72             # Rate limiting: 10-second cooldown between
alert dispatches
73             if current_time - last_alert_time > 10:
74                 now_str = time.strftime('%H:%M:%S')
75                 motion_message = (
76                     f"*SILENTSENSE ALERT*\n\n"
77                     f"*Time:* {now_str}\n"
78                     f"*Light Status:* ON (LED Turned ON)\n"
n"
79                     f"*Motion Status:* No motion for 30s!"
"
80                 )
81                 send_telegram_alert(motion_message)
82                 last_alert_time = current_time
83
84                 last_motion_state = motion_detected
85

```

```

86         except Exception as e:
87             print(f"App error: {e}", flush=True)
88
89             time.sleep(0.2)
90
91 if __name__ == "__main__":
92     main()

```

Listing 6.2: Python Logic Controller & Telegram Gateway

6.3 Functionality and Parameter Matrix

Tables 6.1 and 6.2 define the method contracts for firmware and application modules.

Table 6.1: Embedded C++ Method Specifications

Function Name	Return Type	Parameters	Description
get_motion()	int	None	Returns 1 if motion active, 0 if inactive
get_panic()	int	None	Returns 1 if Panic threshold breached. Returns 1 if Panic Pin D2 is driven LOW, else 0.
setup()	void	None	Initializes GPIO modes, Bridge functions, and stabilizing delay.
loop()	void	None	Polling hardware pins and adjusting output state continuously.

Table 6.2: Python Application Module Functions

Function Name	Return Type	Parameters	Description
send_telegram_alert()	None	message_text (str)	Transmits JSON payload to Telegram API.
main()	None	None	Polling loop managing state shifts and triggering alerts.

Chapter 7

Conclusion & Future Scope

7.1 Conclusion

The **SilentSense** project successfully demonstrates a low-cost, real-time safety monitoring solution designed for vulnerable individuals in domestic environments. By pairing micro-controller hardware processing with a Python network bridge, the system achieves instant local hardware reaction times alongside remote cloud alert dispatching.

Key project outcomes include:

1. **Privacy-Preserving Safety Monitoring:** Effective monitoring without cameras or invasive video processing.
2. **Dual Alert Modes:** Rapid response via explicit manual panic triggers alongside automatic inactivity monitoring.
3. **Instant Telegram Alerts:** Real-time push notification delivery directly to caregivers' devices over Wi-Fi.

7.2 Future Scope

- **Multi-Room Array Expansion:** Deploying an interconnected mesh network of PIR nodes to track house-wide movement transitions.
- **Machine Learning Behavior Profiling:** Integrating adaptive thresholds that automatically learn the individual's regular sleep and waking schedules to reduce false inactivity alarms.
- **Heart-rate & Radar Sensing:** Incorporating mmWave Doppler radar sensors to detect micro-movements, such as breathing, during sleep.
- **Battery Backup Integration:** Adding an uninterrupted power supply (UPS) circuit to ensure uninterrupted operation during power outages.

Bibliography

- [1] R. Suresh, J. K. Patel, and M. Sharma, “PIR and Sensor Network Integration for In-activity Monitoring in Smart Homes,” *IEEE Transactions on Consumer Electronics*, vol. 67, no. 3, pp. 215–223, 2021.
- [2] O. Banos, C. Villalonga, and M. Damas, “Design and Implementation of Ambient Assisted Living Monitoring Architectures,” *IEEE Sensors Journal*, vol. 16, no. 15, pp. 6001–6010, 2016.
- [3] Telegram Bot API Documentation. Available online: <https://core.telegram.org/bots/api> (Accessed: August 2026).
- [4] Arduino Bridge Library Reference. Available online: <https://www.arduino.cc/en/Reference/YunBridgeLibrary> (Accessed: August 2026).
- [5] S. Patel, H. Park, and P. Bonato, “A review of wearable sensors and systems with application in rehabilitation,” *Journal of NeuroEngineering and Rehabilitation*, vol. 9, no. 21, pp. 1–17, 2012.